CONTENTSPHERE

Infrastructure Automation Using Ansible

Ansible Fundamentals Mini Project Report

Project	ContentSphere Infrastructure Automation
Prepared by	Saad Momin
Course / Context	IT Vedant – Ansible Fundamentals Mini Project
Primary tools	Ansible, Ubuntu Linux, Nginx, AWS EC2, AWS S3
AWS Region used	ap-south-1
GitHub repository	github.com/saad7804/Ansible-Project

This report documents the implementation and verification work completed during the project. Screenshots are intentionally not fabricated; supporting screenshots can be added to the project's **screenshots/** directory separately.

Prepared: 25 October 2026

Contents

- 1. Project Overview
- 2. Objectives
- 3. Tools and Environment
- 4. Project Structure
- 5. Implementation
- 6. AWS Automation
- 7. Verification and Results
- 8. Security and Operational Considerations
- 9. Challenges and Resolutions
- 10. Conclusion
- Appendix A – Execution Commands

Note: This report reflects the implementation details and command outputs described during the project workflow. Where a result was not directly captured as a screenshot in this report, it is described as a reported verification rather than as attached evidence.

1. Project Overview

ContentSphere is a demonstration web application used to practice configuration management and AWS automation with Ansible. The project configures an Ubuntu managed host, deploys a webpage through Nginx, uses Jinja2 templates and variables, organizes tasks into a reusable role, and automates selected AWS resources through Ansible modules.

The project separates the control node, which runs Ansible, from the managed Ubuntu EC2 host. AWS automation tasks run locally on the control node and use the AWS identity available to that node. The project uses an IAM role for AWS API authentication rather than storing long-term AWS access keys in project files.

2. Objectives

- Use Ansible ad-hoc commands to inspect and manage a Linux server.
- Install, start, stop, restart, and verify the Nginx service.
- Create directories, files, permissions, and a Linux user on the managed host.
- Deploy the ContentSphere webpage with an Ansible playbook.
- Use group variables and Jinja2 templates for HTML and Nginx configuration.
- Create and execute a reusable Ansible role with a handler.
- Provision an EC2 instance and create an S3 bucket with an uploaded test object.
- Document the implementation and publish project source files in GitHub.

3. Tools and Environment

Component	Use in project
Ansible	Ad-hoc operations, playbooks, roles, templates, and handlers.
Ubuntu Linux	Control node and managed web-server operating environment.

Component	Use in project
Nginx	Serves the ContentSphere webpage over HTTP on port 80.
Jinja2	Renders the webpage and Nginx site configuration from variables.
AWS EC2	Managed web-server host and an additional instance created by Task 6.
AWS S3	Stores the ContentSphere test object.
amazon.aws collection	Provides Ansible modules for AWS EC2 and S3 tasks.
Boto3 / Botocore	AWS SDK dependencies used by the AWS collection.
Git / GitHub	Version control and source-code publication.

Environment details

AWS Region: **ap-south-1**. Managed host private IP: **172.31.46.14**. The control node used an attached IAM role named **admin_role** for AWS API calls. The managed-host inventory was kept local and excluded from Git tracking; a sanitized example inventory was added to the repository.

4. Project Structure

```
ContentSphere-Ansible-Project/
■■■ README.md
■■■ .gitignore
■■■ inventory                # local only; ignored by Git
■■■ inventory.example        # sanitized example
■■■ setup-webserver.yml
■■■ deploy-webpage.yml
■■■ run-webserver-role.yml
■■■ group_vars/
■   ■■■ webserver.yml
■■■ handlers/
■   ■■■ main.yml
■■■ templates/
■   ■■■ index.html.j2
■   ■■■ contentsphere.conf.j2
■■■ roles/
■   ■■■ webserver/
■       ■■■ tasks/main.yml
■       ■■■ handlers/main.yml
■       ■■■ vars/main.yml
■       ■■■ templates/
■■■ playbooks/
■   ■■■ ec2.yml
■   ■■■ s3.yml
■■■ files/
■   ■■■ contentsphere-test.txt
■■■ screenshots/             # evidence to be added
■■■ documentation/
    ■■■ Project_Report.pdf
```

The root-level playbooks cover basic server setup and template-based deployment. The reusable role contains its own tasks, handlers, variables, and templates. AWS resource automation is kept in the **playbooks/** directory.

5. Implementation

5.1 Ad-hoc commands and basic server management

Ad-hoc Ansible commands were used to inspect server information and connectivity, install Nginx, manage its service state, create an application directory and information file, apply file permissions, and ensure the Linux user **ansibleuser** exists.

```
ansible -i inventory webserver -m ping
ansible -i inventory webserver -b -m command -a "uptime"
ansible -i inventory webserver -b -m apt -a "name=nginx state=present update_cache=yes"
ansible -i inventory webserver -b -m service -a "name=nginx state=started enabled=yes"
```

5.2 Playbook-based web server setup

setup-webserver.yml installs Nginx, ensures the service is enabled and running, creates **/opt/ansible-demo**, writes an information file, and ensures the demo user exists. This demonstrates declarative configuration through repeatable tasks.

5.3 Variables and Jinja2 templates

The **group_vars/webserver.yml** file defines the application name, application directory, and web port. The **templates/index.html.j2** template renders the page title and body using those variables. The **templates/contentsphere.conf.j2** template renders the Nginx server configuration.

```
app_name: "ContentSphere Platform"
app_directory: "/var/www/html"
web_port: 80
```

The deployment playbook installs Nginx, creates the application directory, deploys the rendered webpage, installs the Nginx site configuration, enables the site, removes the default site, validates the Nginx configuration with **nginx -t**, and ensures Nginx is running. A handler restarts Nginx when the site configuration changes.

5.4 Reusable webserver role

The **roles/webserver** role packages the Nginx installation and configuration tasks into a reusable unit. Its tasks deploy the templates, enable the ContentSphere site, remove the default site, validate the configuration, and manage the service. The handler restarts Nginx when notified. The role playbook is **run-webserver-role.yml**.

The role was executed successfully. A subsequent run reported **changed=0**, indicating that the tasks were idempotent for the checked state.

6. AWS Automation

6.1 EC2 provisioning

The playbook **playbooks/ec2.yml** runs on localhost and uses the **amazon.aws.ec2_ami_info** module to find an available Ubuntu 24.04 LTS AMI from the official Ubuntu owner ID. It sorts the returned images by creation date and launches one **t3.micro** instance in **ap-south-1**. The instance is tagged with the project name and **ManagedBy=Ansible**. Public IP assignment is disabled in the playbook.

Property	Reported result
Instance ID	i-0453f19c86fa74aa1
Instance type	t3.micro
Region	ap-south-1
State after playbook	running
Public IP assignment	disabled by playbook

The instance was reported as running after the playbook completed. Confirm its current state in the AWS Console before any demonstration or cleanup, because cloud resource state can change after the recorded run.

6.2 S3 bucket and object upload

The playbook **playbooks/s3.yml** creates an S3 bucket in **ap-south-1**, uploads **files/contentsphere-test.txt** as **contentsphere-test.txt**, and verifies the object using the AWS collection. AWS CLI was also used to list the object and retrieve its contents.

Property	Reported result
Bucket	contentsphere-ansible-888869353635-1792942322
Object key	contentsphere-test.txt
Object size	70 bytes
Verification	Object listed and content retrieved successfully

The bucket name is supplied to the playbook through an environment variable, **CONTENTSPHERE_BUCKET_NAME**, so a future user can provide their own bucket name. The bucket was intentionally retained for instructor review. Do not delete it without confirming that evaluation is complete.

7. Verification and Results

Area	Verification performed / reported outcome
Managed host	Ad-hoc connectivity and server information checks were run.
Nginx	Service state was confirmed active/running; service management commands were tested.
Webpage	HTTP request to localhost returned HTML with the ContentSphere Platform title and heading.
Jinja2 variables	Rendered page displayed the application directory and port values.
Nginx configuration	The playbook ran nginx -t to validate configuration syntax.
Role	Role execution succeeded; repeat run was idempotent (changed=0).
EC2	Provisioning playbook reported a running t3.micro instance in ap-south-1.
S3	Bucket and object were listed; AWS CLI retrieved the uploaded test-file contents.
GitHub	Project was pushed to the main branch of saad7804/Ansible-Project.

Evidence screenshots should be captured separately and added under **screenshots/**. Suggested evidence includes Ansible connectivity, ad-hoc command output, playbook execution, the rendered webpage, role execution, EC2 instance details, and the S3 bucket/object. This PDF does not embed screenshots that were not supplied.

8. Security and Operational Considerations

- Use an IAM role or another secure credential mechanism for AWS API access; do not store long-term AWS secret keys in source code.
- The actual inventory file contains host connection details and a private-key path. It is excluded from Git; inventory.example uses placeholders.
- Never commit private SSH keys, AWS credential files, tokens, passwords, or other secrets.
- Grant the AWS identity only the permissions required by the playbooks in a production environment; an administrative role is broader than ideal for least privilege.
- The EC2 playbook disables public IP assignment. Verify subnet, routing, and security-group configuration separately if remote access is required.
- EC2 instances and other AWS resources can incur charges while they exist. Review the resource in the AWS Console after taking evidence.
- The S3 bucket is being retained for review. Confirm with the instructor before deleting it or its object.

9. Challenges and Resolutions

Challenge	Resolution
AWS CLI and AWS collection dependencies	Installed AWS CLI, amazon.aws collection, Boto3, and Botocore on the control node.
AWS authentication	Used the attached control-node IAM role instead of hardcoding AWS access keys.
AMI selection	Queried official Ubuntu AMIs and selected the latest result by creation date.
S3 playbook directory	Created the required playbooks/ and files/ directories before using the playbooks.
Role idempotency	Re-ran the role and observed changed=0 on the repeat execution.
GitHub rejected password authentication	Used a valid GitHub authentication method and successfully pushed the main branch.
Avoid publishing local connection details	Added inventory to .gitignore and created inventory.example for repository users.

10. Conclusion

The project demonstrates core Ansible configuration-management techniques, including ad-hoc commands, YAML playbooks, variables, Jinja2 templates, handlers, and reusable roles. It also demonstrates AWS resource automation for EC2 and S3 and verifies the created S3 object using both Ansible and AWS CLI.

The project source is published in GitHub at **https://github.com/saad7804/Ansible-Project**. To complete the submission, add genuine screenshots to the **screenshots/** directory and confirm the current status of AWS resources before final evaluation.

Appendix A – Execution Commands

Run these commands from the project directory on the Ansible control node. The AWS S3 command requires the environment variable to be set first.

```
cd ~/ContentSphere-Ansible-Project

# Test managed-host connectivity
ansible -i inventory webserver -m ping

# Configure the managed web server
ansible-playbook -i inventory setup-webserver.yml

# Deploy ContentSphere with variables and templates
ansible-playbook -i inventory deploy-webpage.yml

# Execute the reusable role
ansible-playbook -i inventory run-webserver-role.yml

# Provision the additional EC2 instance
ansible-playbook playbooks/ec2.yml

# Create the S3 bucket and upload the test file
export CONTENTSPHERE_BUCKET_NAME="your-unique-bucket-name"
ansible-playbook playbooks/s3.yml

# Verify the S3 object
aws s3 ls "s3://$CONTENTSPHERE_BUCKET_NAME/" --region ap-south-1
aws s3 cp "s3://$CONTENTSPHERE_BUCKET_NAME/contentsphere-test.txt" - --region ap-south-1
```

End of report. Replace example bucket values as needed and capture screenshots from the actual environment for the final submission.